BudgetApp Intermediate

Katalog kodu źródłowego

Aplikacja konsolowa w C# / .NET 7.0 z Entity Framework Core i SQLite

Projekt edukacyjny — wzorce: Repository, Observer, Extension methods, SOLID/ISP

by **Dominik Wiśniewski**

PLIKI	JĘZYK	FRAMEWORK	ORM	LINIE KODU
20	C#	.NET 7.0	EF Core	~890

Wygenerowano: 2026-05-11

Struktura projektu

20 plików ułożonych w 5 warstwach: Punkt wejścia, Data, Entity, Logging, Repositories.

#01	Program.cs	ENTRY POINT
#02	BudgetAppIntermediate.csproj	CONFIG
#03	BudgetAppIntermediateContext.cs	EF CORE
#04	BudgetAppIntermediateSqliteContext.cs	EF CORE
#05	IEntity.cs	INTERFACE
#06	EntityBase.cs	ABSTRACT
#07	BillBase.cs	BASE
#08	OneTimeBill.cs	DOMAIN
#09	RegularBill.cs	DOMAIN
#10	UnRegularBill.cs	DOMAIN
#11	EntityExtensions.cs	EXTENSION
#12	IEventRepository.cs	INTERFACE
#13	BillEventArgs.cs	EVENT
#14	AuditLogger.cs	LOGGER
#15	IReadRepository.cs	INTERFACE
#16	IWriteRepository.cs	INTERFACE
#17	IRepository.cs	INTERFACE
#18	FileRepository.cs	IMPL
#19	SqliteRepository.cs	IMPL
#20	RepositoryExtensions.cs	EXTENSION

ENTRY POINT

Program.cs

Punkt wejścia aplikacji

src/Program.cs

Główna pętla programu w stylu top-level statements (.NET 7). Pozwala wybrać sposób zapisu danych (SQLite albo plik tekstowy) i obsługuje pełen CRUD na rachunkach poprzez interaktywne menu. Podpina trzy handlery zdarzeń repozytorium: ItemAdded, ItemRemoved i AllItemRemoved.

Program.cs — linie 1-24 / 465

```
1  using BudgetAppIntermediate.Data;
2  using BudgetAppIntermediate.Entity;
3  using BudgetAppIntermediate.Repositories;
4  using BudgetAppIntermediate.Repositories.Extensions;
5  using Microsoft.EntityFrameworkCore;
6
7  string optionMainMenu = null;
8  string option = null;
9
10 var oneTimeBills = new List<OneTimeBill>();
11
12 Console.WriteLine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData));
13
14 Console.WriteLine("==========================");
15 Console.WriteLine("Welcome to The Budget App!");
16 Console.WriteLine("==========================\n");
17
18 while (optionMainMenu != "1" && optionMainMenu != "2")
19 {
20     Console.WriteLine("Choose where do you want to save your data: ");
21     Console.WriteLine("[1] Save in the database");
22     Console.WriteLine("[2] Save in the text file");
23     Console.WriteLine("[q] Exit the app");
24
```

ENTRY POINT

Program.cs

linie 25-59 / 465

```
25      optionMainMenu = Console.ReadLine();
26
27      if (optionMainMenu == "q")
28      {
29          break;
30      }
31
32      switch (optionMainMenu)
33      {
34          case "1":
35
36              var sqliteRepository = new SqliteRepository<OneTimeBill>(new BudgetAppIntermediateSqliteContext());
37              var auditLoggerSqlite = new AuditLogger<SqliteRepository<OneTimeBill>>(sqliteRepository);
38
39              sqliteRepository.ItemAdded += BillRepositoryOnItemAdded;
40              sqliteRepository.ItemRemoved += BillRepositoryItemRemoved;
41              sqliteRepository.AllItemRemoved += BillRepositoryAllItemRemoved;
42
43              while (true)
44              {
45                  Console.WriteLine("<<< MENU >>>");
46                  Console.WriteLine("[1] Add One Time Bills");
47                  Console.WriteLine("[2] Add Regular Bills");
48                  Console.WriteLine("[3] Add Unregular Bills");
49                  Console.WriteLine("---");
50                  Console.WriteLine("[4] Display Bills");
51                  Console.WriteLine("---");
52                  Console.WriteLine("[5] Delete Specific Bill by Id");
53                  Console.WriteLine("[6] Delete All Bill");
54                  Console.WriteLine("---");
55                  Console.WriteLine("---");
56                  Console.WriteLine("[q] if want to exit the app\n");
57
58                  option = Console.ReadLine();
59
```

Program.cs

linie 60-94 / 465

```
60                  if(option == "q")
61                  {
62                      break;
63                  }
64
65                  switch (option)
66                  {
67                      case "1": //ADD ONE TIME BILLS //PRZETESTOWANE
68                          while (true)
69                          {
70                              OneTimeBill oneTimeBill = new OneTimeBill();
71
72                              Console.WriteLine("Give a name of the bill: ");
73                              oneTimeBill.Name = Console.ReadLine();
74
75                              Console.WriteLine("Give the price of the bill: ");
76                              string priceStr = Console.ReadLine();
77
78                              try
79                              {
80                                  oneTimeBill.Price = decimal.Parse(priceStr);
81                              }
82                              catch (Exception e)
83                              {
84                                  Console.WriteLine($"Exteption caught: you have to provide a number, not a symbol or text.");
85                              }
86
87                              DateTime currentDate = DateTime.Now;
88                              oneTimeBill.Date = currentDate.ToString("dd-MM-yyyy");
89
90                              oneTimeBills.Add(oneTimeBill);
91
92                              while (true)
93                              {
94                                  Console.WriteLine("Do you want to add another One Time Bill?");
```

ENTRY POINT

Program.cs

linie 95-129 / 465

```
95                                  Console.WriteLine("\n[y] to add new bill");
96                                  Console.WriteLine("[n] if want to come back to the menu\n");
97                                  option = Console.ReadLine();
98
99                                  if (option != "y" && option != "n")
100                                 {
101                                     Console.WriteLine("You have to enter correct letter\n");
102                                 }
103                                 else
104                                 {
105                                     break;
106                                 }
107                             }
108
109                             if (option == "n")
110                             {
111                                 sqliteRepository.addBatch(oneTimeBills);
112                                 oneTimeBills.Clear();
113                                 break;
114                             }
115                         }
116
117
118                         break;
119
120                     case "2": // ADD REGULAR BILLS
121
122                         Console.WriteLine("\nStill in process, please use only One Time Bills in this version\n");
123
124                         break;
125
126                     case "3": // ADD UNREGULAR BILLS
127
128                         Console.WriteLine("\nStill in process, please use only One Time Bills in this version\n");
129
```

ENTRY POINT

Program.cs

linie 130-164 / 465

```
130                         break;
131
132                     case "4":  //DISPLAY BILLS // PRZETESTOWANE
133
134                         Console.WriteLine();
135                         try
136                         {
137                             IEnumerable<OneTimeBill> allBills = sqliteRepository.GetAll();
138
139                             Console.WriteLine("============================================================");
140
141                             foreach (var item in allBills)
142                             {
143                                 Console.WriteLine($"Id: {item.Id} Name: {item.Name}, Price: {item.Price} pln, Date: {item.Date}");
144                             }
145
146                             Console.WriteLine("============================================================\n");
147                         }
148                         catch (Exception e)
149                         {
150                             Console.WriteLine($"Exception caught: {e.Message}\n");
151                         }
152
153                         break;
154
155                     case "5": //DELETE SPECIFIC BILLS BY ID // PRZETESTOWANE
156
157                         Console.WriteLine();
158
159                         try
160                         {
161                             IEnumerable<OneTimeBill> allBillsId = sqliteRepository.GetAll();
162
163                             Console.WriteLine("============================================================");
164
```

Program.cs

linie 165-199 / 465

```
165                         foreach (var item in allBillsId)
166                         {
167                             Console.WriteLine($"Id: {item.Id} Name: {item.Name}, Price: {item.Price} pln, Date: {item.Date}");
168                         }
169
170                         Console.WriteLine("==========================================================");
171
172                         Console.WriteLine("\n['id number'] to delete the bill");
173                         Console.WriteLine("---");
174                         Console.WriteLine("[c] if want to come back to the menu\n");
175                         option = Console.ReadLine();
176
177                         if (option == "c")
178                         {
179                             break;
180                         }
181                         else
182                         {
183                             try
184                             {
185                                 int id = int.Parse(option);
186                                 sqliteRepository.RemoveById(id);
187                             }
188                             catch (Exception)
189                             {
190                                 Console.WriteLine($"Exception caught: you have to provide a number, not a symbol or text.");
191                             }
192                         }
193                     }
194                     catch (Exception e)
195                     {
196                         Console.WriteLine($"Exception caught: {e.Message}");
197                     }
198
199                     Console.WriteLine();
```

ENTRY POINT

Program.cs

linie 200-234 / 465

```
200
201                         break;
202
203                     case "6": //DELETE ALL BILLS // PRZETESTOWANE
204
205                         Console.WriteLine();
206                         Console.WriteLine("Are you sure to delete all bills?\n");
207                         Console.WriteLine("[y] yes\n");
208                         Console.WriteLine("[n] no\n");
209
210                         option = Console.ReadLine();
211
212                         if (option == "y")
213                         {
214                             try
215                             {
216                                 sqliteRepository.RemoveAll();
217                             }
218                             catch (Exception e)
219                             {
220                                 Console.WriteLine($"\nException catched: {e.Message}");
221                             }
222                         }
223                         else if (option == "n")
224                         {
225                             break;
226                         }
227
228                         Console.WriteLine();
229
230                         break;
231
232                     default:
233                         Console.WriteLine("Try to use correct sign from MENU list");
234                         break;
```

ENTRY POINT

Program.cs

linie 235-269 / 465

```
235                 }
236             }
237
238             break;
239
240         case "2":
241
242             var fileRepository = new FileRepository<OneTimeBill>();
243             var auditLogger = new AuditLogger<FileRepository<OneTimeBill>>(fileRepository);
244
245             fileRepository.ItemAdded += BillRepositoryOnItemAdded;
246             fileRepository.ItemRemoved += BillRepositoryItemRemoved;
247             fileRepository.AllItemRemoved += BillRepositoryAllItemRemoved;
248
249             while (true)
250             {
251                 Console.WriteLine("<<< MENU >>>");
252                 Console.WriteLine("[1] Add One Time Bills");
253                 Console.WriteLine("[2] Add Regular Bills");
254                 Console.WriteLine("[3] Add Unregular Bills");
255                 Console.WriteLine("---");
256                 Console.WriteLine("[4] Display Bills");
257                 Console.WriteLine("---");
258                 Console.WriteLine("[5] Delete Specific Bill by Id");
259                 Console.WriteLine("[6] Delete All Bill");
260                 Console.WriteLine("---");
261                 Console.WriteLine("---");
262                 Console.WriteLine("[q] if want to exit the app\n");
263
264                 option = Console.ReadLine();
265
266                 if (option == "q")
267                 {
268                     break;
269                 }
```

Program.cs

linie 270-304 / 465

```
270
271                 switch (option)
272                 {
273                     case "1": //ADD ONE TIME BILLS //PRZETESTOWANE
274
275                         while (true)
276                         {
277                             OneTimeBill oneTimeBill = new OneTimeBill();
278
279                             Console.WriteLine("Give a name of the bill: ");
280                             oneTimeBill.Name = Console.ReadLine();
281
282                             Console.WriteLine("Give the price of the bill: ");
283                             string priceStr = Console.ReadLine();
284
285                             try
286                             {
287                                 oneTimeBill.Price = decimal.Parse(priceStr);
288                             }
289                             catch (Exception e)
290                             {
291                                 Console.WriteLine($"Exteption caught: you have to provide a number, not a symbol or text.");
292                             }
293
294                             DateTime currentDate = DateTime.Now;
295                             oneTimeBill.Date = currentDate.ToString("dd-MM-yyyy");
296
297                             oneTimeBills.Add(oneTimeBill);
298
299                             while (true)
300                             {
301                                 Console.WriteLine("Do you want to add another One Time Bill?");
302                                 Console.WriteLine("\n[y] to add new bill");
303                                 Console.WriteLine("[n] if want to come back to the menu\n");
304                                 option = Console.ReadLine();
```

ENTRY POINT

Program.cs

linie 305-339 / 465

```
305
306                             if (option != "y" && option != "n")
307                             {
308                                 Console.WriteLine("You have to enter correct letter\n");
309                             }
310                             else
311                             {
312                                 break;
313                             }
314                         }
315
316                         if (option == "n")
317                         {
318                             fileRepository.addBatch(oneTimeBills);
319                             oneTimeBills.Clear();
320                             break;
321                         }
322                     }
323
324                     break;
325
326                 case "2": // ADD REGULAR BILLS
327
328                     Console.WriteLine("\nStill in process, please use only One Time Bills in this version\n");
329
330                     break;
331
332                 case "3": // ADD UNREGULAR BILLS
333
334                     Console.WriteLine("\nStill in process, please use only One Time Bills in this version\n");
335
336                     break;
337
338                 case "4": //DISPLAY BILLS // PRZETESTOWANE
339
```

Program.cs

linie 340-374 / 465

```
340                         Console.WriteLine();
341                         try
342                         {
343                             IEnumerable<OneTimeBill> allBills = fileRepository.GetAll();
344
345                             Console.WriteLine("============================================================");
346
347                             foreach (var item in allBills)
348                             {
349                                 Console.WriteLine($" {item.Id} Name: {item.Name}, Price: {item.Price} pln, Date: {item.Date}");
350                             }
351
352                             Console.WriteLine("============================================================\n");
353                         }
354                         catch (Exception e)
355                         {
356                             Console.WriteLine($"Exception caught: {e.Message}\n");
357                         }
358
359                         break;
360
361                     case "5": //DELETE SPECIFIC BILLS BY ID // PRZETESTOWANE
362
363                         Console.WriteLine();
364
365                         try
366                         {
367                             IEnumerable<OneTimeBill> allBillsId = fileRepository.GetAll();
368
369                             Console.WriteLine("============================================================");
370                             foreach (var item in allBillsId)
371                             {
372                                 Console.WriteLine($"Id: {item.Id} Name: {item.Name}, Price: {item.Price} pln, Date: {item.Date}");
373                             }
374                             Console.WriteLine("============================================================");
```

Program.cs

linie 375-409 / 465

```
375
376                         Console.WriteLine("\n['id number'] to delete the bill");
377                         Console.WriteLine("---");
378                         Console.WriteLine("[c] if want to come back to the menu\n");
379                         option = Console.ReadLine();
380
381                         if (option == "c")
382                         {
383                             break;
384                         }
385                         else
386                         {
387                             try
388                             {
389                                 int id = int.Parse(option);
390                                 fileRepository.RemoveById(id);
391                             }
392                             catch (Exception)
393                             {
394                                 Console.WriteLine($"Exception caught: you have to provide a number, not a symbol or text.");
395                             }
396                         }
397                     }
398                     catch (Exception e)
399                     {
400                         Console.WriteLine($"Exception caught: {e.Message}");
401                     }
402
403                     Console.WriteLine();
404
405                     break;
406
407                 case "6": //DELETE ALL BILLS // PRZETESTOWANE
408
409                     Console.WriteLine();
```

ENTRY POINT

Program.cs

linie 410-444 / 465

```
410                         Console.WriteLine("Are you sure to delete all bills?\n");
411                         Console.WriteLine("[y] yes\n");
412                         Console.WriteLine("[n] no\n");
413                         option = Console.ReadLine();
414                         if (option == "y")
415                         {
416                             try
417                             {
418                                 fileRepository.RemoveAll();
419                             }
420                             catch (Exception e)
421                             {
422                                 Console.WriteLine($"\nException catched: {e.Message}");
423                             }
424                         }
425                         else if (option == "n")
426                         {
427                             break;
428                         }
429
430                         Console.WriteLine();
431
432                         break;
433
434                     default:
435                         Console.WriteLine("Try to use correct sign from MENU list");
436                         break;
437                 }
438             }
439
440             break;
441
442         case "q":
443
444             optionMainMenu = "q";
```

ENTRY POINT

Program.cs

linie 445-465 / 465

```
445
446             break;
447
448         default:
449             Console.WriteLine("Try to use correct sign from suggest options");
450             break;
451     }
452 }
453
454 static void BillRepositoryOnItemAdded(object? sender, BillEventArgs<OneTimeBill> e)
455 {
456     Console.WriteLine($"\nBill added => {e.Item.Name} with action {e.Action} from {sender?.GetType().Name}\n");
457 }
458 static void BillRepositoryItemRemoved(object? sender, BillEventArgs<OneTimeBill> e)
459     {
460         Console.WriteLine($"\nBill removed {e.Item.Name} with action {e.Action} from {sender?.GetType().Name}\n");
461     }
462 static void BillRepositoryAllItemRemoved(object? sender, BillEventArgs<OneTimeBill> e)
463 {
464     Console.WriteLine($"\n{e.Action}: All bills have been removed!\n");
465 }
```

CONFIG

BudgetAppIntermediate.csproj

Plik projektu .NET

src/BudgetAppIntermediate.csproj

Konfiguracja MSBuild dla projektu. Target framework: net7.0, OutputType: Exe. Lista pakietów: EF Core (InMemory + Sqlite + Tools) oraz Newtonsoft.Json.

BudgetAppIntermediate.csproj — linie 1-20 / 20

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net7.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="7.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="7.0.13" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="7.0.13">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.3" />
  </ItemGroup>

</Project>
```

EF CORE

BudgetAppIntermediateContext.cs

Kontekst EF Core (in-memory)

src/Data/BudgetAppIntermediateContext.cs

Klasa kontekstu EF Core używająca bazy in-memory („StorageAppDb"). Definiuje DbSet<BillBase> i pełni rolę alternatywnego/testowego kontekstu. Produkcyjnie używany jest wariant SQLite — ten plik to bardziej szkielet do eksperymentów.

BudgetAppIntermediateContext.cs — linie 1-21 / 21

```
1  using BudgetAppIntermediate.Entity;
2  using Microsoft.EntityFrameworkCore;
3
4  namespace BudgetAppIntermediate.Data
5  {
6      public class BudgetAppIntermediateContext : DbContext
7      {
8          public DbSet<BillBase> billBase => Set<BillBase>();
9
10         //public DbSet<OneTimeBill> oneTimeBills => Set<OneTimeBill>();
11
12         //public DbSet<RegularBills> regularBills => Set<RegularBills>();
13
14         //We have to give the name of database
15         protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
16         {
17             base.OnConfiguring(optionsBuilder);
18             optionsBuilder.UseInMemoryDatabase("StorageAppDb");
19         }
20     }
21 }
```

EF CORE

BudgetAppIntermediateSqliteContext.cs

Kontekst EF Core (SQLite)

src/Data/BudgetAppIntermediateSqliteContext.cs

Produkcyjny kontekst EF Core oparty o SQLite — trzyma OneTimeBills w pliku budget.db. Ścieżka do bazy ustalana jest dynamicznie z bieżącego katalogu roboczego. Konfiguracja połączenia przez UseSqlite z parametrem Data Source.

BudgetAppIntermediateSqliteContext.cs — linie 1-20 / 20

```
1  using BudgetAppIntermediate.Entity;
2  using Microsoft.EntityFrameworkCore;
3
4  public class BudgetAppIntermediateSqliteContext : DbContext
5  {
6      public DbSet<OneTimeBill> OneTimeBills { get; set; }
7
8      public string DbPath { get; }
9      public BudgetAppIntermediateSqliteContext()
10     {
11         // var folder = Environment.SpecialFolder.LocalApplicationData;
12         // var path = Environment.GetFolderPath(folder);
13         // DbPath = System.IO.Path.Join(path, "budget.db");
14         var path = Directory.GetCurrentDirectory();
15         DbPath = Path.Combine(path, "budget.db");
16     }
17
18     protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
19         => optionsBuilder.UseSqlite($"Data Source={DbPath}");
20 }
```

INTERFACE

IEntity.cs

Bazowy interfejs encji

src/Entity/IEntity.cs

Wspólny kontrakt wszystkich obiektów domenowych — wymaga obecności pola Id. Używany jako ograniczenie generyczne w repozytoriach (where T : IEntity), dzięki czemu te same operacje działają na różnych typach encji.

IEntity.cs — linie 1-7 / 7

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public interface IEntity
4     {
5         int Id { get; set; }
6     }
7 }
```

ABSTRACT

EntityBase.cs

Abstrakcyjna baza encji

src/Entity/EntityBase.cs

Abstrakcyjna klasa bazowa implementująca IEntity. Standaryzuje pole Id we wszystkich encjach domenowych. Pierwszy poziom hierarchii dziedziczenia — z niej wywodzi się BillBase.

EntityBase.cs — linie 1-7 / 7

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public abstract class EntityBase : IEntity
4     {
5         public int Id { get; set; }
6     }
7 }
```

BASE

BillBase.cs

Bazowa klasa rachunku

src/Entity/BillBase.cs

Wspólna baza dla wszystkich typów rachunków — dziedziczy po EntityBase. Dodaje pola Name (nazwa) i Price (kwota typu decimal). Drugi poziom dziedziczenia: EntityBase → BillBase → konkretne rachunki.

BillBase.cs — linie 1-8 / 8

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public class BillBase : EntityBase
4     {
5         public string? Name { get; set; }
6         public decimal Price { get; set; }
7     }
8 }
```

DOMAIN

OneTimeBill.cs

Rachunek jednorazowy

src/Entity/OneTimeBill.cs

Reprezentuje pojedynczy rachunek z konkretną datą — najprostszy typ w aplikacji. Rozszerza BillBase o pole Date (string w formacie dd-MM-yyyy). Jedyny typ rachunku, który jest w pełni obsługiwany w obecnej wersji menu.

OneTimeBill.cs — linie 1-9 / 9

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public  class OneTimeBill : BillBase
4     {
5         public string Date { get; set; }
6
7         public override string ToString() => $"Id: {Id}, Name of Bill: {Name}, Executed: {Price} pln";
8     }
9 }
```

DOMAIN

RegularBill.cs

Rachunek regularny

src/Entity/RegularBill.cs

Reprezentuje rachunek cykliczny (np. abonament). Dodaje pole FixedFrequency — stałą częstotliwość naliczania. W obecnej wersji menu zaznaczony jako „still in process".

RegularBill.cs — linie 1-9 / 9

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public class RegularBill : BillBase
4     {
5         public int FixedFrequency { get; set; }
6
7         public override string ToString() => $"Id: {Id}, Name of Bill: {Name}, Executed: {Price} pln, Frequency: {FixedFrequency}";
8     }
9 }
```

DOMAIN

UnRegularBill.cs

Rachunek nieregularny

src/Entity/UnRegularBill.cs

Dziedziczy po RegularBill i dodaje własne pole Frequency. Przeznaczony do modelowania opłat nieregularnych z dodatkowym parametrem. Również w fazie „in progress" — nie jest jeszcze podpięty do menu.

UnRegularBill.cs — linie 1-9 / 9

```
1 namespace BudgetAppIntermediate.Entity
2 {
3     public class UnRegularBill : RegularBill
4     {
5         public int Frequency { get; set; }
6
7         public override string ToString() => $"Id: {Id}, Name of Bill: {Name}, Bill size: {Price}, Frequency: {Frequency}";
8     }
9 }
```

EXTENSION

EntityExtensions.cs

Rozszerzenia encji

src/Entity/Extensions/EntityExtensions.cs

Statyczna klasa rozszerzeń dla typów implementujących IEntity. Udostępnia metodę Copy<T>() — głębokie kopiowanie obiektu przez JSON. Pożyteczna, gdy potrzeba duplikatu odciętego od oryginalnej referencji.

EntityExtensions.cs — linie 1-13 / 13

```
1  using System.Text.Json;
2
3  namespace BudgetAppIntermediate.Entity.Extensions
4  {
5      public static class EntityExtensions
6      {
7          public static T? Copy<T>(this T itemToCopy) where T : IEntity
8          {
9              var json = JsonSerializer.Serialize<T>(itemToCopy);
10             return JsonSerializer.Deserialize<T>(json);
11         }
12     }
13 }
```

INTERFACE

IEventRepository.cs

Kontrakt zdarzeniowy

src/Logging/IEventRepository.cs

Interfejs definiujący trzy zdarzenia repozytorium: ItemAdded, ItemRemoved, AllItemRemoved. Pozwala odłączyć logikę powiadamiania od konkretnej implementacji repozytorium. Klasyczna implementacja wzorca Observer z użyciem natywnych zdarzeń C#.

IEventRepository.cs — linie 1-8 / 8

```
1 using BudgetAppIntermediate.Entity;
2
3 public interface IEventRepository<T> where T : IEntity, new()
4 {
5     event EventHandler<BillEventArgs<T>> ItemAdded;
6     event EventHandler<BillEventArgs<T>> ItemRemoved;
7     event EventHandler<BillEventArgs<T>> AllItemRemoved;
8 }
```

EVENT

BillEventArgs.cs

Argumenty zdarzenia

src/Logging/BillEventArgs.cs

Generyczna klasa argumentów zdarzenia dziedzicząca po EventArgs. Przenosi nazwę akcji (np. „BillAdded") oraz obiekt, którego zdarzenie dotyczy. Wymaga, by typ T implementował IEntity.

BillEventArgs.cs — linie 1-14 / 14

```
1  using BudgetAppIntermediate.Entity;
2
3  public class BillEventArgs<T> : EventArgs
4      where T : IEntity
5  {
6      public string Action { get; set; }
7      public T Item { get; set; }
8
9      public BillEventArgs(string action, T item)
10     {
11         Action = action;
12         Item = item;
13     }
14 }
```

LOGGER

AuditLogger.cs

Logger audytowy

src/Logging/AuditLogger.cs

Loguje każdą operację repozytorium (dodanie, usunięcie, masowe usunięcie) do pliku audits.txt. Subskrybuje zdarzenia w konstruktorze, więc logowanie odbywa się w pełni automatycznie. Każdy wpis ma znacznik czasu i opis akcji.

AuditLogger.cs — linie 1-24 / 48

```
1  using BudgetAppIntermediate.Entity;
2  using BudgetAppIntermediate.Repositories;
3
4  public class AuditLogger<T>
5      where T: class, IEventRepository<OneTimeBill>
6  {
7      private static string _logFilePath = "audits.txt";
8
9      public AuditLogger(T repository)
10     {
11         repository.ItemRemoved += LogEvent;
12         repository.ItemAdded += LogEvent;
13         repository.AllItemRemoved += LogEventNonParameters;
14     }
15
16     private void LogEvent(object? sender, BillEventArgs<OneTimeBill> e)
17     {
18         WriteLog($"{e.Action}-{e.Item.Name}");
19     }
20
21     private void LogEventNonParameters(object sender, EventArgs e)
22     {
23         WriteLog($"AllBillsDeleted");
24     }
```

AuditLogger.cs

linie 25-48 / 48

```
25
26      private void WriteLog(string logEntry)
27      {
28          try
29          {
30              string timestamp = DateTime.Now.ToString("yyy-MM-ddTHH:mm:ss.fffffff");
31              string logMessage = $"[{timestamp}]-{logEntry}";
32              string fullPath = Path.GetFullPath(_logFilePath);
33
34              Console.WriteLine($"Writing to log file at: {fullPath}");
35
36              using (var writer = new StreamWriter(fullPath, true))
37              {
38                  writer.WriteLine(logMessage);
39              }
40          }
41          catch (Exception e)
42          {
43              Console.WriteLine($"Failed to write to log file: {e.Message}");
44          }
45
46      }
47
48  }
```

INTERFACE

IReadRepository.cs

Interfejs odczytu

src/Repositories/IReadRepository.cs

Interfejs odpowiedzialny wyłącznie za operacje odczytu (GetAll, GetById). Realizacja zasady ISP z SOLID — rozdzielenie odpowiedzialności od zapisu. Parametr generyczny używa kowariancji (out T), umożliwiając rzutowanie między pochodnymi.

IReadRepository.cs — linie 1-10 / 10

```
1  using BudgetAppIntermediate.Entity;
2
3  namespace BudgetAppIntermediate.Repositories
4  {
5      public interface IReadRepository<out T> where T : class, IEntity
6      {
7          IEnumerable<T> GetAll();
8          T? GetById(int id);
9      }
10 }
```

INTERFACE

IWriteRepository.cs

Interfejs zapisu

src/Repositories/IWriteRepository.cs

Interfejs odpowiedzialny tylko za operacje zapisu (Add, RemoveAll, RemoveById, Save). Komplement do IReadRepository — razem realizują pełen CRUD. Parametr generyczny używa kontrawariancji (in T).

IWriteRepository.cs — linie 1-13 / 13

```
1  using BudgetAppIntermediate.Entity;
2
3  namespace BudgetAppIntermediate.Repositories
4  {
5      public interface IWriteRepository<in T> where T : class, IEntity
6      {
7          void Add(T item);
8          //void Remove(T item);
9          void RemoveAll();
10         void RemoveById(int id);
11         void Save();
12     }
13 }
```

INTERFACE

IRepository.cs

Pełny kontrakt repo

src/Repositories/IRepository.cs

Kompozytowy interfejs łączący IReadRepository i IWriteRepository. Pełen kontrakt repozytorium dla implementacji, które obsługują zarówno odczyt jak i zapis. Stanowi szczyt hierarchii interfejsów warstwy dostępu do danych.

IRepository.cs — linie 1-9 / 9

```
1 using BudgetAppIntermediate.Entity;
2
3 namespace BudgetAppIntermediate.Repositories
4 {
5     public interface IRepository<T> : IReadRepository<T>, IWriteRepository<T>
6         where T : class, IEntity
7     {
8     }
9 }
```

IMPL

FileRepository.cs

Repozytorium plikowe (JSON)

src/Repositories/FileRepository.cs

Implementacja repozytorium zapisująca dane do pliku bills.txt w formacie JSON. Po starcie wczytuje istniejące rekordy i odtwarza Id, a operacje zapisu serializują kolekcję ponownie. Wystawia zdarzenia (IEventRepository), więc współpracuje z AuditLoggerem.

FileRepository.cs — linie 1-24 / 106

```
1  using BudgetAppIntermediate.Entity;
2  using System.Text.Json;
3
4  namespace BudgetAppIntermediate.Repositories
5  {
6      public class FileRepository<T> : IRepository<T>, IEventRepository<T>
7          where T : BillBase, new()
8      {
9          private List<T> _items;
10         public static readonly List<string> _actions = new List<string>
11         {
12             "BillAdded",
13             "BillDeleted",
14             "AllBillsDeleted"
15         };
16
17         private readonly string _fileName = "bills.txt";
18
19         public event EventHandler<BillEventArgs<T>> ItemAdded;
20         public event EventHandler<BillEventArgs<T>> ItemRemoved;
21         public event EventHandler<BillEventArgs<T>> AllItemRemoved;
22
23         public FileRepository()
24         {
```

IMPL

FileRepository.cs

linie 25-59 / 106

```
25            _items = new List<T>();
26
27            this.LoadDataFromFile();
28        }
29
30        private void LoadDataFromFile()
31        {
32            if (File.Exists(_fileName))
33            {
34                string json = File.ReadAllText(_fileName);
35
36                if(!string.IsNullOrWhiteSpace(json))
37                {
38                    _items = JsonSerializer.Deserialize<List<T>>(json);
39                }
40            }
41
42            for (int id = 0; id < _items.Count; id++)
43            {
44                _items[id].Id = id + 1;
45            }
46        }
47
48        public IEnumerable<T> GetAll()
49        {
50            if (_items.Count > 0)
51            {
52                this.LoadDataFromFile();
53                return _items.ToList();
54            }
55            else
56            {
57                throw new Exception("Failure - the list of the bills is empty");
58            }
59        }
```

IMPL

FileRepository.cs

linie 60-94 / 106

```
60
61          public T GetById(int Id)
62          {
63              return _items.Single(item => item.Id == Id);
64          }
65
66          public void Add(T item)
67          {
68              item.Id = _items.Count + 1;
69              _items.Add(item);
70              this.ItemAdded?.Invoke(this, new BillEventArgs<T>("BillAdded", item));
71          }
72
73          public void RemoveById(int id)
74          {
75              var itemToRemove = _items.FirstOrDefault(item => item.Id == id);
76
77              if (itemToRemove != null)
78              {
79                  _items.Remove(itemToRemove);
80                  this.ItemRemoved?.Invoke(this, new BillEventArgs<T>("BillRemoved", itemToRemove));
81                  this.Save();
82              }
83          }
84
85          public void RemoveAll()
86          {
87              if (_items.Count > 0)
88              {
89                  _items.Clear();
90                  this.AllItemRemoved?.Invoke(this, new BillEventArgs<T>("AllBillsDeleted", null));
91                  this.Save();
92              }
93              else
94              {
```

IMPL

FileRepository.cs

linie 95-106 / 106

```
95                 throw new Exception("Failure - the list of the bills is already empty");
96             }
97         }
98
99         public void Save()
100        {
101            var json = JsonSerializer.Serialize(_items);
102
103            File.WriteAllText(_fileName, json);
104        }
105    }
106 }
```

IMPL

SqliteRepository.cs

Repozytorium SQLite (EF Core)

src/Repositories/SqliteRepository.cs

Implementacja repozytorium oparta o Entity Framework Core i DbContext. Operuje na DbSet<T> i obsługuje pełen CRUD (Find, Add, Remove, RemoveRange). Po każdej operacji zmieniającej stan wyzwala odpowiednie zdarzenie.

SqliteRepository.cs — linie 1-24 / 74

```
using BudgetAppIntermediate.Entity;
using BudgetAppIntermediate.Repositories;
using Microsoft.EntityFrameworkCore;

public class SqliteRepository<T> : IRepository<T>, IEventRepository<T>
    where T : class, IEntity, new()
{
    private readonly DbContext _dbContext;
    private readonly DbSet<T> _dbSet;

    private List<T> _items;

    public SqliteRepository(DbContext dbContext)
    {
        _dbContext = dbContext;
        _dbSet = _dbContext.Set<T>();
    }

    public event EventHandler<BillEventArgs<T>> ItemAdded;
    public event EventHandler<BillEventArgs<T>> ItemRemoved;
    public event EventHandler<BillEventArgs<T>> AllItemRemoved;

    public IEnumerable<T> GetAll()
    {
```

IMPL

SqliteRepository.cs

linie 25-59 / 74

```
25          return _dbSet.ToList();
26      }
27
28      public T? GetById(int id)
29      {
30          return _dbSet.Find(id);
31      }
32
33      public void Add(T item)
34      {
35          _dbSet.Add(item);
36          this.ItemAdded?.Invoke(this, new BillEventArgs<T>("BillAdded", item));
37      }
38
39      public void RemoveById(int id)
40      {
41          var itemToRemove = _dbSet.Find(id);
42          if (itemToRemove != null)
43          {
44              _dbSet.Remove(itemToRemove);
45              _dbContext.SaveChanges();
46              this.ItemRemoved?.Invoke(this, new BillEventArgs<T>("BillRemoved", itemToRemove));
47          }
48          else
49          {
50              throw new Exception("Failure: there is no such an item to remove.");
51          }
52      }
53
54      public void RemoveAll()
55      {
56          var allBills = this.GetAll();
57
58          if(allBills.Any())
59          {
```

IMPL

SqliteRepository.cs

linie 60-74 / 74

```
60            _dbSet.RemoveRange(allBills);
61            _dbContext.SaveChanges();
62            this.AllItemRemoved?.Invoke(this, new BillEventArgs<T>("AllBillsDeleted", null));
63        }
64        else
65        {
66            throw new Exception("Failure: the list of the bills is already empty");
67        }
68    }
69
70    public void Save()
71    {
72        _dbContext.SaveChanges();
73    }
74 }
```

EXTENSION

RepositoryExtensions.cs

Rozszerzenia repozytorium

src/Repositories/Extensions/RepositoryExtensions.cs

Statyczna klasa rozszerzeń dla IRepository<T>. Udostępnia metodę addBatch — dodaje wszystkie elementy z listy i wywołuje Save() na końcu. Eliminuje duplikację pętli „dodaj + zapisz" w Program.cs.

RepositoryExtensions.cs — linie 1-18 / 18

```
1  using BudgetAppIntermediate.Entity;
2
3  namespace BudgetAppIntermediate.Repositories.Extensions
4  {
5      public static class RepositoryExtensions
6      {
7          public static void addBatch<T>(this IRepository<T> repository, List<T> items)
8              where T : class, IEntity
9          {
10             foreach (var bill in items)
11             {
12                 repository.Add(bill);
13             }
14
15             repository.Save();
16         }
17     }
18 }
```

Co robi aplikacja i czemu służy

FUNKCJA

BudgetAppIntermediate to konsolowa ewidencja rachunków domowych. Po starcie wybierasz sposób zapisu danych (baza SQLite albo plik tekstowy), a potem z menu dodajesz nowe rachunki, wyświetlasz listę, usuwasz pojedynczy wpis po Id lub czyścisz całą historię.

ARCHITEKTURA

Kod podzielono na 3 warstwy: Entity (model domenowy z hierarchią dziedziczenia), Repositories (warstwa dostępu do danych z separacją odczyt/zapis) oraz Logging (audyt operacji oparty o zdarzenia). Wykorzystano wzorce Repository, Observer, Extension methods i generyki z constraints.

WARTOŚĆ EDUKACYJNA

Aplikacja prezentuje kluczowe mechanizmy C#: dziedziczenie i polimorfizm, interfejsy z wariancją (out / in), zdarzenia (events i EventHandler<T>), Entity Framework Core (DbContext, DbSet, migracje), serializację JSON oraz metody rozszerzające.

DLA KOGO

Projekt portfolio na poziomie 'intermediate' — idealny do nauki lub jako baza do rozbudowy. Dwie alternatywne implementacje IRepository (plikowa i SQLite) doskonale ilustrują, dlaczego warto programować pod interfejs, a nie pod konkretną klasę.

PLATFORMA	ORM	BAZA	FORMAT PLIKU	LOG	APP
.NET 7.0	EF Core 7	SQLite	JSON	audits.txt	Console / Top-level